Hash

Prof. Henrique Batista da Silva

Transformação de Chave (*Hashing*)

- A busca por uma chave ocorre sempre através de comparações entre a chave de pesquisa e a chave na tabela.
- Alternativamente, pode-se buscar uma chave em uma tabela por meio do cálculo de sua posição.
- Para isso, utiliza-se uma função (fução hash).

Transformação de Chave (*Hashing*)

- A função hash calcula a posição em que o registro se encontra na tabela.

- Assim, os registros armazenados em uma tabela são diretamente endereçados a partir de uma função (transformação aritmética) sobre a chave de pesquisa.

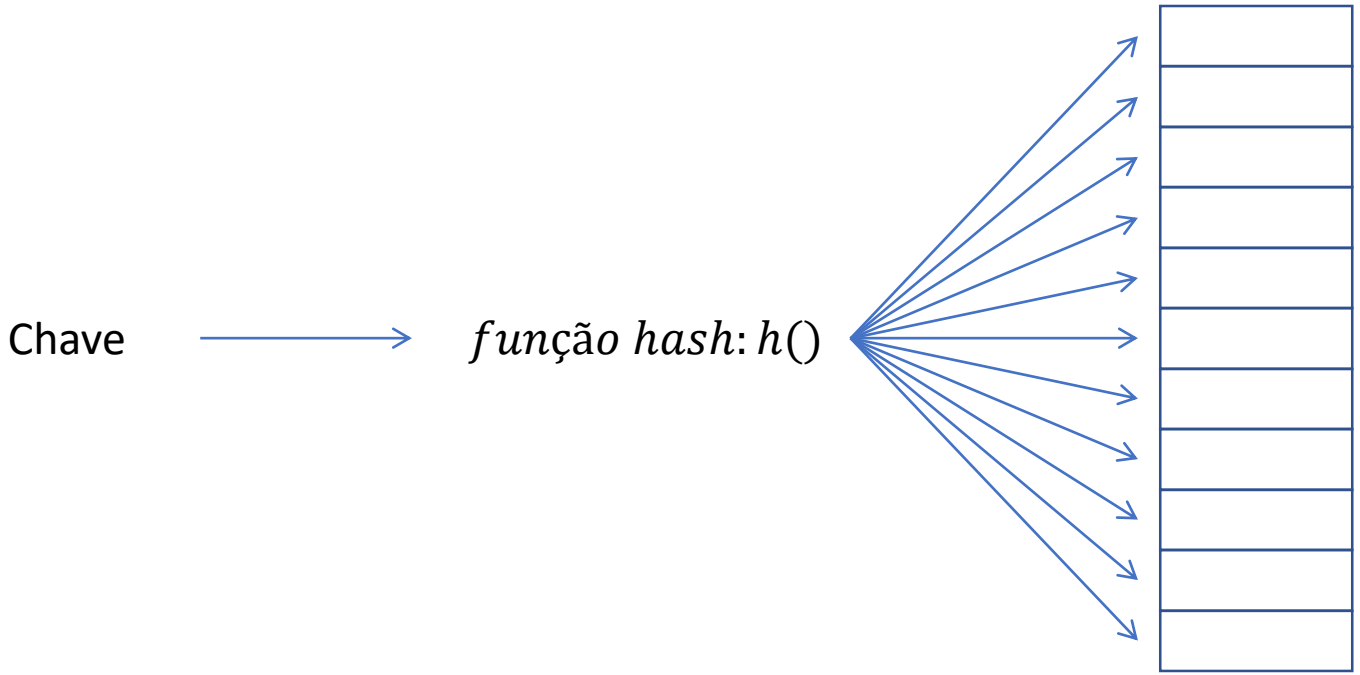
Transformação de Chave (*Hashing*)

- Na busca sequencial, temos tempo: (n).

- Na busca binária, temos tempo: $(\log n)$.

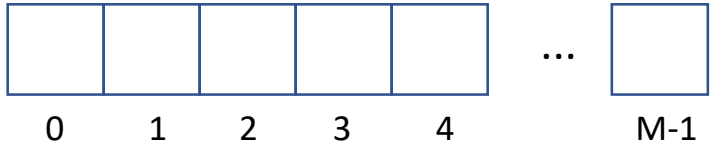
- Na tabela hash, temos tempo: (1).
  - A busca na tabela hash, sempre retorna a posição da tabela da chave na tabela.

Transformação de Chave (*Hashing*)



Transformação de Chave (*Hashing*)

- Motivação:
- Suponha: Tabela com M posições, em que:
  - N Chaves de valor menor que M
  - Se N $\leq$ M: Ok



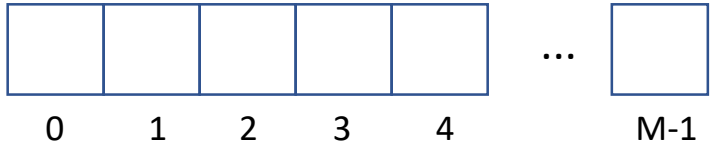
Transformação de Chave (*Hashing*)

Inserção das chaves {2, 3, 4, 6, 10} em uma tabela de tamanho 11.

0	
1	
2	2
3	3
4	4
5	
6	6
7	
8	
9	
10	10

Transformação de Chave (*Hashing*)

- Motivação:
- Suponha: Tabela com M posições, em que:
  - N Chaves de valor menor que M



  - Se $N \leq M$: Ok
  - Se $N > M$: ???

Transformação de Chave (*Hashing*)

- Um método de pesquisa com o uso da transformação de chave é constituído de duas etapas principais:
  - Calcular o valor da **função de tansformação (função hash)**, a qual transforma a chave de pesquisa em um endereço da tabela.
    - Hash(*Chave*) = posição na tabela.

  - Considerando que duas ou mais chaves podem ser transformadas em um mesmo endereço de tabela, é necessário existir um método para lidar com **colisões**.

Transformação de Chave (*Hashing*)

- Fatalmente algumas **colisões** irão ocorrer:
  - colisões têm de ser resolvidas de alguma forma.

- Mesmo com uma função de transformação que distribua os registros de forma uniforme entre as entradas da tabela, existe uma alta probabilidade de haver colisões.

Transformação de Chave (*Hashing*)

- Paradoxo do aniversário (Feller, 69):
  - Em um grupo de 23 ou mais pessoas, juntas ao acaso, existe uma chance maior do que 50% de que 2 pessoas comemorem aniversário no mesmo dia.
  - Assim, se for utilizada uma função de transformação uniforme que enderece 23 chaves randômicas em uma tabela de tamanho 365, a probabilidade de que haja **colisões** é maior do que 50%.

Função Hash

- Uma função de transformação deve mapear chaves em inteiros dentro do intervalo $[0 \ldots M - 1]$, onde M é o tamanho da tabela.
  - h(*x*) = pos. na tabela
- Características
  - Seja simples de ser computada (Eficiência)
  - Para cada chave de entrada, qualquer uma das saídas possíveis é igualmente provável de ocorrer (Uniformidade).

Função Hash

- Como as transformações sobre as chaves são aritméticas, deve-se transformar as chaves não-numéricas em números.

Método da Divisão

- Usa o resto da divisão por M:
  - $h(k) = k \; mod \; M$.
  - onde k é um inteiro correspondente à chave.

- Fácil e eficiente.

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28} em uma tabela de tamanho 11. Função $h(x) = x \bmod M$.

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28} em uma tabela de tamanho 11. Função $h(x) = x \; mod \; M$.

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	10

H(10) = 10 mod 11.

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28} em uma tabela de tamanho 11. Função $h(x) = x \; mod \; M$.

0		
1		
2	2	H(2) = 2 mod 11.
3		
4		
5		
6		
7		
8		
9		
10	10	H(10) = 10 mod 11.

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28} em uma tabela de tamanho 11. Função $h(x) = x \; mod \; M$.

0		
1		
2	2	H(2) = 2 mod 11.
3	3	H(3) = 3 mod 11.
4		
5		
6		
7		
8		
9		
10	10	H(10) = 10 mod 11.

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28} em uma tabela de tamanho 11. Função $h(x) = x \; mod \; M$.

0		
1		
2	2	H(2) = 2 mod 11.
3	3	H(3) = 3 mod 11.
4	15	H(15) = 15 mod 11.
5		
6		
7		
8		
9		
10	10	H(10) = 10 mod 11.

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28} em uma tabela de tamanho 11. Função $h(x) = x \; mod \; M$.

0		
1		
2	2	H(2) = 2 mod 11.
3	3	H(3) = 3 mod 11.
4	15	H(15) = 15 mod 11.
5		
6	28	H(28) = 28 mod 11.
7		
8		
9		
10	10	H(10) = 10 mod 11.

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28} em uma tabela de tamanho 11. Função $h(x) = x \; mod \; M$.

O que acontecerá ao tentar inserir novo elemento: 21

0		
1		
2	2	H(2) = 2 mod 11.
3	3	H(3) = 3 mod 11.
4	15	H(15) = 15 mod 11.
5		
6	28	H(28) = 28 mod 11.
7		
8		
9		
10	10	H(10) = 10 mod 11.

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28} em uma tabela de tamanho 11. Função $h(x) = x \; mod \; M$.

O que acontecerá ao tentar inserir novo elemento: 21

Colisão! Como resolver?

0		
1		
2	2	H(2) = 2 mod 11.
3	3	H(3) = 3 mod 11.
4	15	H(15) = 15 mod 11.
5		
6	28	H(28) = 28 mod 11.
7		
8		
9		
10	10	H(10) = 10 mod 11.

H(21) = 21 mod 11. →

Listas Encadeadas

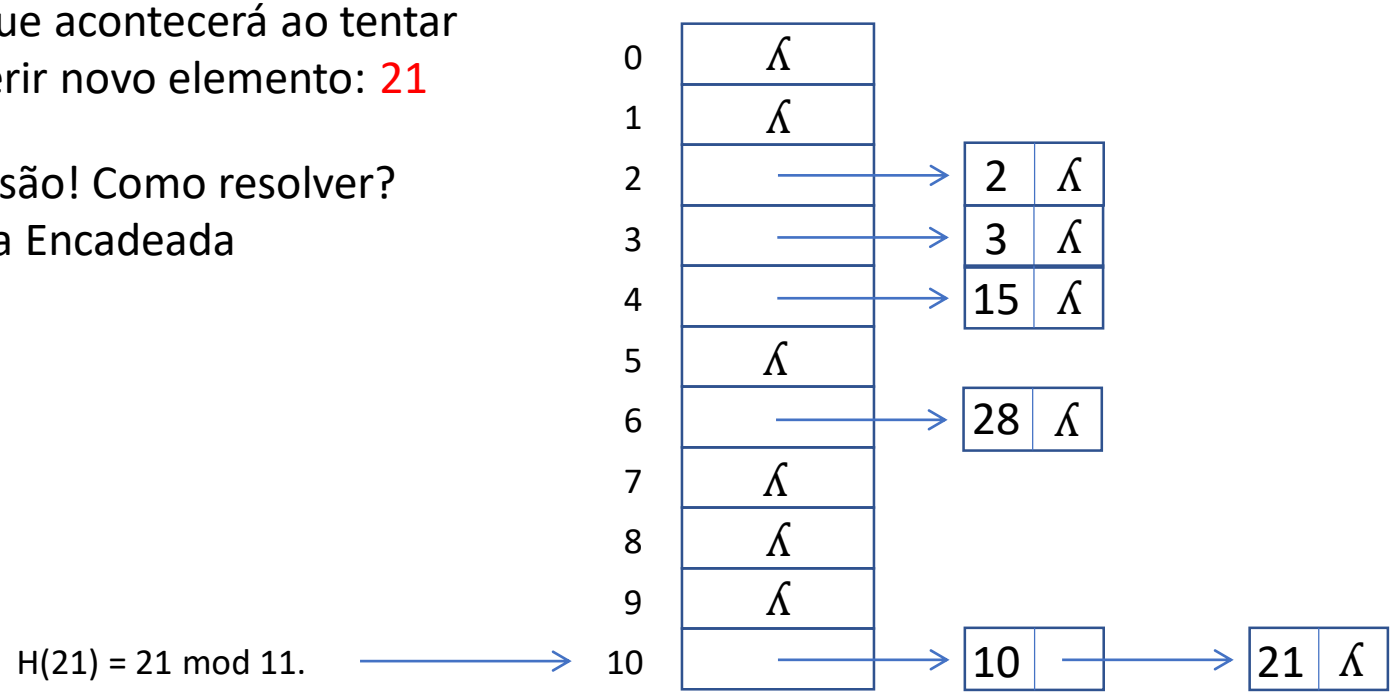
- Construir uma lista linear encadeada para cada endereço da tabela.
- Assim, todas as chaves com mesmo endereço são encadeadas em uma lista linear.

Listas Encadeadas

Inserção das chaves {10, 2, 3, 15, 28} em uma tabela de tamanho 11. Função h(x) = x mod M.

O que acontecerá ao tentar inserir novo elemento: 21

Colisão! Como resolver?
Lista Encadeada

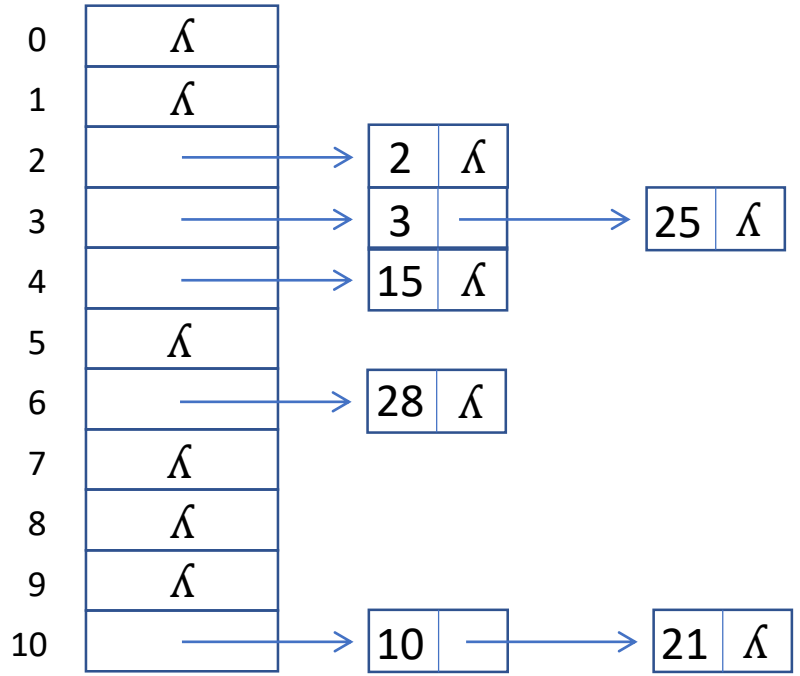


Listas Encadeadas

Inserção das chaves {10, 2, 3, 15, 28} em uma tabela de tamanho 11. Função h(x) = x mod M.

O que acontecerá ao tentar inserir novo elemento: 25

H(25) = 25 mod 11.

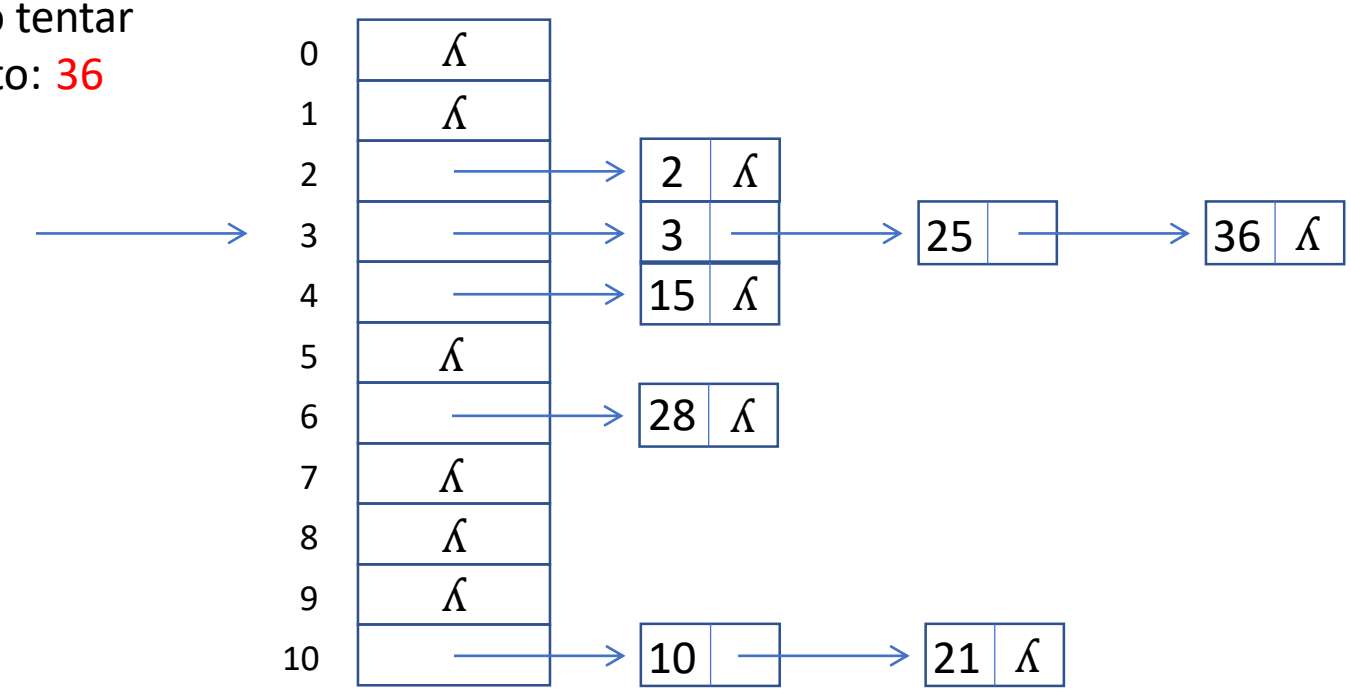


Listas Encadeadas

Inserção das chaves {10, 2, 3, 15, 28} em uma tabela de tamanho 11. Função h(x) = x mod M.

O que acontecerá ao tentar inserir novo elemento: 36

H(36) = 36 mod 11.



Estrutura e operações da tabela com listas encadeadas

- O código da tabela hash utiliza a TAD Lista auto referenciada, que é uma lista encadeada ensinada na aula de listas lineares.

- Esta tabela utiliza como elemento chave o próprio item (composto de números inteiros positivos).

Análise

- Assumindo que qualquer item do conjunto tem igual probabilidade de ser endereçado para qualquer entrada da tabela:
  - Comprimento esperado de cada lista: N/M,
    - N representa o número de registros e M o tamanho da tabela.
  - Para pesquisa, inserir e retirar elementos:
    - $(1 + \frac{N}{M})$ operações em média. 1 é a constante que representa o tempo para encontrar e entrada na tabela.

Análise

- Melhor caso é (1):
  - Elemento encontrado na primeira comparação (não sendo necessário percorrer a lista).
- Pior caso é (n):
  - Todos os elementos ficam na mesma chave da tabela (apenas uma lista encadeada com todos os elementos).
  - Necessário percorrer toda a lista.
- Caso médio é $(1 + \frac{N}{M})$:
  - Elementos estão distribuídos uniformemente pela tabela (litas de mesmo tamanho).

Endereçamento Aberto

- Quando o número de registros a serem armazenados na tabela puder ser previamente estimado, então não haverá necessidade de usar listas encadeadas.
- Pode-se utilizar os lugares vazios na própria tabela para resolver as **colisões** (para $M > N$).
- Todas as chaves são armazenadas na própria tabela.

Endereçamento Aberto

- Proposta (*hashing* linear):
  - A posição h_j na tabela é dada por:
  - $h_j = (h(x) + j)\ mod\ M$, para $1 \leq j \leq M\ - 1$.
  - Caso haja colisões, haverá uma sequencia de tentativas, em que $j = 1, 2, \ldots, M - 1$.

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28, 21} em uma tabela de tamanho 11. Função h(x) = x mod M.

Para resolver colisões:

$$h_j = (h(x) + j)\ mod\ M,$$
$$para\ 1 \leq j \leq M\ - 1$$

0	
1	
2	2
3	3
4	15
5	
6	28
7	
8	
9	
10	10

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28, 21} em uma tabela de tamanho 11. Função h(x) = x mod M.

Para resolver colisões:

$$h_j = (h(x) + j)\ mod\ M,$$
$$para\ 1 \leq j \leq M\ - 1$$

H(21) = 21 mod 11.

0	
1	
2	2
3	3
4	15
5	
6	28
7	
8	
9	
10	10

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28, 21} em uma tabela de tamanho 11. Função h(x) = x mod M.

Para resolver colisões:

$$h_j = (h(x) + j)\ mod\ M,$$
$$para\ 1 \leq j \leq M\ - 1$$

0	21
1	
2	2
3	3
4	15
5	
6	28
7	
8	
9	
10	10

$h_1 = (h(21) + 1)\ mod\ 11$

H(21) = 21 mod 11.

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28, 21, 25} em uma tabela de tamanho 11. Função h(x) = x mod M.

Para resolver colisões:

$$h_j = (h(x) + j)\ mod\ M,$$
$$para\ 1 \leq j \leq M\ - 1$$

H(21) = 25 mod 11.

0	21
1	
2	2
3	3
4	15
5	
6	28
7	
8	
9	
10	10

$$h_1 = (h(21) + 1)\ mod\ 11$$

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28, 21, 25} em uma tabela de tamanho 11. Função h(x) = x mod M.

Para resolver colisões:

$$h_j = (h(x) + j)\ mod\ M,$$
$$para\ 1 \leq j \leq M\ - 1$$

H(21) = 25 mod 11.

0	21	$h_1 = (h(21) + 1)\ mod\ 11$
1		
2	2	
3	3	
4	15	$h_1 = (h(25) + 1)\ mod\ 11$
5		
6	28	
7		
8		
9		
10	10	

Inserção de elementos

Inserção das chaves {10, 2, 3, 15, 28, 21, 25} em uma tabela de tamanho 11. Função h(x) = x mod M.

Para resolver colisões:

$$h_j = (h(x) + j)\ mod\ M,$$
$$para\ 1 \leq j \leq M\ - 1$$

H(21) = 25 mod 11.

0	21	$h_1 = (h(21) + 1)\ mod\ 11$
1		
2	2	
3	3	
4	15	$h_1 = (h(25) + 1)\ mod\ 11$
5	25	$h_2 = (h(25) + 2)\ mod\ 11$
6	28	
7		
8		
9		
10	10	

Análise

- Seja $\alpha = N/M$ (fator de carga).
  - No melhor caso, tem-se: (1)
    - Elemento encontrado na primeira comparação.
  - No pior caso, tem-se: (n)
    - É necessário comparar o elemento com todas as chaves da tabela até encontrar a ultima posição vazia.
  - No caso médio, tem-se: $\left(\frac{1}{1-\alpha}\right)$.
    - Ou seja, para $N = 5\ e\ M = 10, temos\ \alpha = 0{,}5$. Gerando 2 comparação no caso médio.

Exercícios

- Represente a tabela *hash* obtida a partir da inserção de elementos com as seguintes chaves: 19 / 17 / 35 / 10 / 12 / 3.
  - Caso seja utilizado lista encadeada e a função *hash* seja h(x) = x mod 7.
  - Caso seja utilizado espalhamento linear e a função *hash* seja hj = (h(x) + j) mod 7.

Referências

Material adaptado do livro:

ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.

Os código presentes nos slides estão disponíveis no site do autor (link no último slide)

Referências

- Baseado no material de aulas de:
  - ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.
  - Para downloads dos códigos em Java:
  - http://www2.dcc.ufmg.br/livros/algoritmos-java/implementacoes-03.php

- Outras referências:
  - CORMEN, Thomas H. et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002. 916p. ISBN 8535209263 (Disponível no Acervo).

  - DROZDEK, Adam. Estrutura de dados e algoritmos em C++. São Paulo: Pioneira Thomson Learning, 2002. xviii, 579p. ISBN 8522102953 (Disponível no Acervo).